

window function

window function

分析函数，也称为窗口函数（Window Function），是一种在 SQL 查询中对数据集中的行进行复杂计算的函数。窗口函数的特点在于：它们不会减少查询结果的行数，而是为每一行附加一个新的计算结果。

窗口函数适用于多种数据分析场景，例如计算滚动合计、排名、移动平均、同比环比等。具体的语法说明可参阅 [窗口函数概览](#)。

适用场景

窗口函数主要用于以下数据分析场景：

- 场景：排名与分组；典型问题：「按销售额对每个区域的门店排名」；推荐函数：RANK / DENSE_RANK / ROW_NUMBER / NTILE
- 场景：累计统计；典型问题：「计算每个商品类别按月的累计销售额」；推荐函数：SUM() OVER (... ROWS UNBOUNDED PRECEDING)
- 场景：移动平均；典型问题：「计算门店前后三天的销售移动平均」；推荐函数：AVG() OVER (... ROWS BETWEEN n PRECEDING AND n FOLLOWING)
- 场景：报告分析；典型问题：「找出每年销售额最高的商品类别」；推荐函数：MAX() / SUM() OVER (PARTITION BY ...)
- 场景：行间比较；典型问题：「计算每个类别的同比销售额差异」；推荐函数：LAG / LEAD

快速上手：移动平均示例

下面通过一个完整示例展示如何使用窗口函数计算每个商店「前后三天」的销售移动平均值。

1. 建表与导入数据

```
CREATE TABLE daily_sales (  
  store_id INT,  
  sales_date DATE,  
  sales_amount DECIMAL(10, 2)  
) PROPERTIES ("replication_num" = "1");
```

```
INSERT INTO daily_sales (store_id, sales_date, sales_amount) VALUES
(1, '2023-01-01', 100.00), (1, '2023-01-02', 150.00), (1, '2023-01-03', 200.00),
(1, '2023-01-04', 250.00), (1, '2023-01-05', 300.00), (1, '2023-01-06', 350.00),
(1, '2023-01-07', 400.00), (1, '2023-01-08', 450.00), (1, '2023-01-09', 500.00),
(2, '2023-01-01', 110.00), (2, '2023-01-02', 160.00), (2, '2023-01-03', 210.00),
(2, '2023-01-04', 260.00), (2, '2023-01-05', 310.00), (2, '2023-01-06', 360.00),
(2, '2023-01-07', 410.00), (2, '2023-01-08', 460.00), (2, '2023-01-09', 510.00);
```

2. 编写查询

```
SELECT
    store_id,
    sales_date,
    sales_amount,
    AVG(sales_amount) OVER (
        PARTITION BY store_id
        ORDER BY sales_date
        ROWS BETWEEN 3 PRECEDING AND 3 FOLLOWING
    ) AS moving_avg_sales
FROM daily_sales;
```

3. 查询结果

store_id	sales_date	sales_amount	moving_avg_sales
1	2023-01-01	100.00	175.0000
1	2023-01-02	150.00	200.0000
1	2023-01-03	200.00	225.0000
1	2023-01-04	250.00	250.0000
1	2023-01-05	300.00	300.0000
1	2023-01-06	350.00	350.0000
1	2023-01-07	400.00	375.0000
1	2023-01-08	450.00	400.0000
1	2023-01-09	500.00	425.0000
2	2023-01-01	110.00	185.0000
2	2023-01-02	160.00	210.0000
2	2023-01-03	210.00	235.0000
2	2023-01-04	260.00	260.0000
2	2023-01-05	310.00	310.0000
2	2023-01-06	360.00	360.0000
2	2023-01-07	410.00	385.0000
2	2023-01-08	460.00	410.0000

```
| 2 | 2023-01-09 | 510.00 | 435.0000 |
+-----+-----+-----+-----+
18 rows in set (0.09 sec)
```

基本概念

理解窗口函数前，需要先了解其执行顺序、分区、窗口范围以及当前行这几个核心概念。

处理顺序

使用分析函数的查询，处理过程可以分为三个阶段：

1. 先执行所有的 JOIN、WHERE、GROUP BY 和 HAVING 子句。
2. 将得到的结果集提供给分析函数，并完成所有窗口计算。
3. 如果查询末尾包含 ORDER BY 子句，则在最后处理该子句以得到最终输出顺序。

查询的处理顺序如下图所示：

图片说明：基本概念介绍

结果集分区

分区（Partition）是通过 PARTITION BY 子句定义的逻辑分组。每个分区内的行会被独立计算。

:::caution 注意 分析函数中使用的「分区」与表分区（Table Partition）功能无关。本章中的「分区」仅指与分析函数相关的含义。 :::

窗口

对于分区中的每一行，可以定义一个滑动数据窗口，窗口确定了用于当前行计算所涉及的范围。窗口具有起始行和结束行，根据其定义，窗口可以在一端或两端进行滑动：

- ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW：用于累积和函数，窗口的起始行固定在分区的第一行，结束行从起点一直滑动到分区的最后一行。
- ROWS BETWEEN 3 PRECEDING AND 3 FOLLOWING：用于移动平均值，起点和终点都会随当前行一起滑动。

窗口的大小可以等同于分区中的所有行，也可以仅包含一行。需要注意的是，当窗口靠近分区边界时，由于边界限制，参与计算的行数可能会缩减，此时函数仅返回可用行的计算结果。

在使用窗口函数时，**当前行也会被包含在计算之中**。因此，在处理 n 个项目时，应指定为 (n-1)。例如，要计算 5 天的平均值，窗口应指定为 ROWS BETWEEN 4 PRECEDING AND CURRENT ROW，也可以简写为 ROWS 4 PRECEDING。

当前行

使用分析函数执行的每个计算都基于分区内的当前行（Current Row）。当前行作为确定窗口起始与结束的参考点。

例如 ROWS BETWEEN 6 PRECEDING AND 6 FOLLOWING 定义了一个用于中心移动平均值计算的窗口，该窗口包含当前行、当前行之前的 6 行以及当前行之后的 6 行，共 13 行。

图片说明：当前行

排序函数

排序函数用于在分区内对行进行排序或分组。需要注意：**只有当指定的排序列是唯一值列时，查询结果才是确定的**；如果排序列包含重复值，则每次的查询结果可能不同。更多相关函数可参阅 窗口函数概览。

NTILE 函数

NTILE 用于将查询结果集分成指定数量的桶（组），并为每一行分配一个桶号。在数据分析和报告中常用于分组排序场景。

函数语法

NTILE(num_buckets) **OVER** ([**PARTITION BY** partition_expression] **ORDER BY** order_expression)

参数说明：

- 参数：num_buckets；说明：要将行划分成的桶的数量。
- 参数：PARTITION BY partition_expression；说明：可选。定义如何分区数据。
- 参数：ORDER BY order_expression；说明：必选。定义如何排序数据。

使用示例：将学生按成绩分桶

假设有一张学生考试成绩表 class_student_scores，希望将学生按成绩分成 4 个组，且每组中的学生数量尽可能均匀。

首先创建表并插入数据：

```
CREATE TABLE class_student_scores (  
  class_id INT,
```

```

    student_id INT,
    student_name VARCHAR(50),
    score INT
) DISTRIBUTED BY HASH(student_id) PROPERTIES('replication_num'=1');

```

```
INSERT INTO class_student_scores VALUES
```

```

(1, 1, 'Alice', 85),
(1, 2, 'Bob', 92),
(1, 3, 'Charlie', 87),
(2, 4, 'David', 78),
(2, 5, 'Eve', 95),
(2, 6, 'Frank', 80),
(2, 7, 'Grace', 90),
(2, 8, 'Hannah', 84);

```

然后使用 NTILE 函数按成绩分桶：

```

SELECT
    student_id,
    student_name,
    score,
    NTILE(4) OVER (ORDER BY score DESC) AS bucket
FROM class_student_scores;

```

查询结果：

```

+-----+-----+-----+-----+
| student_id | student_name | score | bucket |
+-----+-----+-----+-----+
| 5 | Eve | 95 | 1 |
| 2 | Bob | 92 | 1 |
| 7 | Grace | 90 | 2 |
| 3 | Charlie | 87 | 2 |
| 1 | Alice | 85 | 3 |
| 8 | Hannah | 84 | 3 |
| 6 | Frank | 80 | 4 |
| 4 | David | 78 | 4 |
+-----+-----+-----+-----+
8 rows in set (0.12 sec)

```

在这个例子中，NTILE(4) 根据成绩将学生分成了 4 个桶，每个桶的学生数量尽可能均匀。

:::caution 注意事项

- 如果不能均匀地将行分配到桶中，某些桶可能会多一行。
- NTILE 在每个分区内独立工作，使用 PARTITION BY 时，每个分区内的数据会分别进行桶分配。 :::

结合 PARTITION BY 使用

如果希望「先按班级分组，再在每个班级内将学生按成绩分成 3 个组」，可以结合 PARTITION BY 使用：

```
SELECT
  class_id,
  student_id,
  student_name,
  score,
  NTILE(3) OVER (PARTITION BY class_id ORDER BY score DESC) AS bucket
FROM class_student_scores;
```

查询结果：

```
+-----+-----+-----+-----+
| class_id | student_id | student_name | score | bucket |
+-----+-----+-----+-----+
| 1 | 2 | Bob | 92 | 1 |
| 1 | 3 | Charlie | 87 | 2 |
| 1 | 1 | Alice | 85 | 3 |
| 2 | 5 | Eve | 95 | 1 |
| 2 | 7 | Grace | 90 | 1 |
| 2 | 8 | Hannah | 84 | 2 |
| 2 | 6 | Frank | 80 | 2 |
| 2 | 4 | David | 78 | 3 |
+-----+-----+-----+-----+
8 rows in set (0.05 sec)
```

可以看到学生按班级进行分区，然后在每个班级内按成绩分成 3 个桶，每个桶的学生数量尽可能均匀。

聚合函数

聚合函数（SUM、AVG、MAX、MIN 等）配合 OVER 子句即可作为窗口函数使用，无需通过 GROUP BY 即可对每行计算分区内的聚合值。

使用 SUM 计算累计值

下面的查询计算 Books 与 Electronics 两个商品类别在 2000 年各月的销售额，以及按月累计的总销售额：

```
SELECT
  i_category,
  year(d_date),
  month(d_date),
  sum(ss_net_paid) AS total_sales,
  sum(sum(ss_net_paid)) OVER (
    PARTITION BY i_category
    ORDER BY year(d_date), month(d_date)
    ROWS UNBOUNDED PRECEDING
  ) AS cum_sales
FROM
  store_sales,
  date_dim d1,
  item
WHERE
  d1.d_date_sk = ss_sold_date_sk
  AND i_item_sk = ss_item_sk
  AND year(d_date) = 2000
  AND i_category IN ('Books', 'Electronics')
GROUP BY
  i_category,
  year(d_date),
  month(d_date);
```

查询结果：

i_category	year(d_date)	month(d_date)	total_sales	cum_sales
Books	2000	1	5348482.88	5348482.88
Books	2000	2	4353162.03	9701644.91
Books	2000	3	4466958.01	14168602.92
Books	2000	4	4495802.19	18664405.11
Books	2000	5	4589913.47	23254318.58
Books	2000	6	4384384.00	27638702.58
Books	2000	7	4488018.76	32126721.34

Books		2000		8		9909227.94		42035949.28	
Books		2000		9		10366110.30		52402059.58	
Books		2000		10		10445320.76		62847380.34	
Books		2000		11		15246901.52		78094281.86	
Books		2000		12		15526630.11		93620911.97	
Electronics		2000		1		5534568.17		5534568.17	
Electronics		2000		2		4472655.10		10007223.27	
Electronics		2000		3		4316942.60		14324165.87	
Electronics		2000		4		4211523.06		18535688.93	
Electronics		2000		5		4723661.00		23259349.93	
Electronics		2000		6		4127773.06		27387122.99	
Electronics		2000		7		4286523.05		31673646.04	
Electronics		2000		8		10004890.96		41678537.00	
Electronics		2000		9		10143665.77		51822202.77	
Electronics		2000		10		10312020.35		62134223.12	
Electronics		2000		11		14696000.54		76830223.66	
Electronics		2000		12		15344441.52		92174665.18	

+-----+-----+-----+-----+-----+

24 rows in set (0.13 sec)

在此示例中，聚合函数 SUM 为每一行定义了一个窗口：起点固定为分区的第一行（UNBOUNDED PRECEDING），终点默认到当前行。这里需要嵌套使用 SUM，因为外层 SUM 是对内层 SUM 的结果再次求和。**嵌套聚合在分析聚合函数中非常常见。**

使用 AVG 计算移动平均值

下面的查询计算 Books 类别 2000 年各月销售额的「3 个月移动平均」（当前月与前两个月）：

```

SELECT
  i_category,
  year(d_date),
  month(d_date),
  sum(ss_net_paid) AS total_sales,
  avg(sum(ss_net_paid)) OVER (
    ORDER BY year(d_date), month(d_date)
    ROWS 2 PRECEDING
  ) AS avg
FROM
  store_sales,
  date_dim d1,

```



```

item
WHERE
  d1.d_date_sk = ss_sold_date_sk
  AND i_item_sk = ss_item_sk
  AND year(d_date) = 2000
  AND i_category = 'Books'
GROUP BY
  i_category,
  year(d_date),
  month(d_date);

```

查询结果：

i_category	year(d_date)	month(d_date)	total_sales	avg
Books	2000	1	5348482.88	5348482.8800
Books	2000	2	4353162.03	4850822.4550
Books	2000	3	4466958.01	4722867.6400
Books	2000	4	4495802.19	4438640.7433
Books	2000	5	4589913.47	4517557.8900
Books	2000	6	4384384.00	4490033.2200
Books	2000	7	4488018.76	4487438.7433
Books	2000	8	9909227.94	6260543.5666
Books	2000	9	10366110.30	8254452.3333
Books	2000	10	10445320.76	10240219.6666
Books	2000	11	15246901.52	12019444.1933
Books	2000	12	15526630.11	13739617.4633

12 rows in set (0.13 sec)

:::caution 注意 输出数据中 avg 列的前两行没有真正按 3 个月的均值计算，因为前面没有足够的行数（SQL 中指定的行数为 3）。 :::

也可以计算「以当前行为中心」的窗口聚合。下面的示例计算 Books 类别 2000 年各月销售额的中心移动平均值，即「前一个月、当前月、后一个月」三个月销售额的平均值：

```

SELECT
  i_category,
  year(d_date),
  month(d_date),
  sum(ss_net_paid) AS total_sales,

```

```

avg(sum(ss_net_paid)) OVER (
  ORDER BY year(d_date), month(d_date)
  ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING
) AS avg_sales
FROM
  store_sales,
  date_dim d1,
  item
WHERE
  d1.d_date_sk = ss_sold_date_sk
  AND i_item_sk = ss_item_sk
  AND year(d_date) = 2000
  AND i_category = 'Books'
GROUP BY
  i_category,
  year(d_date),
  month(d_date);

```

:::caution 注意 输出数据中起始行和结束行的中心移动平均值仅基于两个月的数据计算，因为边界行前后没有足够的行数。 :::

报告函数

报告函数（Reporting Function）的特征是「每一行的窗口范围都是整个分区」。它的主要优势在于能够在一次查询中多次使用同一份数据，从而避免显式 JOIN、提升查询性能。

例如，需求「找出每一年销售额最高的商品类别」即可通过报告函数实现，而无需进行 JOIN：

```

SELECT year, category, total_sum FROM (
  SELECT
    year(d_date) AS year,
    i_category AS category,
    sum(ss_net_paid) AS total_sum,
    max(sum(ss_net_paid)) OVER (PARTITION BY year(d_date)) AS max_sales
  FROM
    store_sales,
    date_dim d1,
    item
  WHERE

```

```

    d1.d_date_sk = ss_sold_date_sk
    AND i_item_sk = ss_item_sk
    AND year(d_date) IN (1998, 1999)
GROUP BY
    year(d_date), i_category
) t
WHERE total_sum = max_sales;

```

内层查询通过 MAX(SUM(ss_net_paid)) 报告出每一年的最高品类销售额，结果如下：

```

+-----+-----+-----+-----+
| year | category | total_sum | max_sales |
+-----+-----+-----+-----+
| 1998 | Electronics | 91723676.27 | 91723676.27 |
| 1998 | Books | 91307909.84 | 91723676.27 |
| 1999 | Electronics | 90310850.54 | 90310850.54 |
| 1999 | Books | 88993351.11 | 90310850.54 |
+-----+-----+-----+-----+
4 rows in set (0.11 sec)

```

外层过滤 total_sum = max_sales 后，得到每年销售额最高的品类：

```

+-----+-----+-----+
| year | category | total_sum |
+-----+-----+-----+
| 1998 | Electronics | 91723676.27 |
| 1999 | Electronics | 90310850.54 |
+-----+-----+-----+
2 rows in set (0.12 sec)

```

报告聚合还可以与嵌套查询结合，解决更复杂的问题。例如「查找产品销售额占其产品类别总销售额 20% 以上的子类别，并从中选出销量最高的 5 种商品」：

```

SELECT i_category AS categ, i_class AS sub_categ, i_item_id
FROM (
    SELECT
        i_item_id, i_class, i_category,
        sum(ss_net_paid) AS sales,
        sum(sum(ss_net_paid)) OVER (PARTITION BY i_category) AS cat_sales,
        sum(sum(ss_net_paid)) OVER (PARTITION BY i_class) AS sub_cat_sales,
        rank() OVER (PARTITION BY i_class ORDER BY sum(ss_net_paid) DESC) AS
rank_in_line

```

```

FROM
    store_sales,
    item
WHERE
    i_item_sk = ss_item_sk
GROUP BY i_class, i_category, i_item_id
) t
WHERE sub_cat_sales > 0.2 * cat_sales AND rank_in_line <= 5;

```

LAG / LEAD 函数

LAG 和 LEAD 函数适用于「行与行之间的比较」场景。两个函数无需自连接即可同时访问表中的多个行，从而显著提升查询效率：

- LAG：访问当前行之前给定偏移处的行。
- LEAD：访问当前行之后给定偏移处的行。

示例 1：使用 LAG 计算同比销售差异

下面的查询希望选取 1999、2000、2001、2002 年中，每个商品类别的总销售额、前一年的总销售额，以及两者之间的差异：

```

SELECT year, category, total_sales, before_year_sales, total_sales - before_year_sales
FROM (
    SELECT
        sum(ss_net_paid) AS total_sales,
        year(d_date) AS year,
        i_category AS category,
        lag(sum(ss_net_paid), 1, 0) OVER (
            PARTITION BY i_category
            ORDER BY YEAR(d_date)
        ) AS before_year_sales
    FROM
        store_sales,
        date_dim d1,
        item
    WHERE
        d1.d_date_sk = ss_sold_date_sk
        AND i_item_sk = ss_item_sk
    GROUP BY
        YEAR(d_date), i_category

```

```
) t
WHERE year IN (1999, 2000, 2001, 2002);
```

查询结果：

```
+-----+-----+-----+-----+-----+
| year | category | total_sales | before_year_sales | (total_sales - before_year_sales) |
+-----+-----+-----+-----+-----+
| 1999 | Books    | 88993351.11 | 91307909.84 | -2314558.73 |
| 2000 | Books    | 93620911.97 | 88993351.11 | 4627560.86 |
| 2001 | Books    | 90640097.99 | 93620911.97 | -2980813.98 |
| 2002 | Books    | 89585515.90 | 90640097.99 | -1054582.09 |
| 1999 | Electronics | 90310850.54 | 91723676.27 | -1412825.73 |
| 2000 | Electronics | 92174665.18 | 90310850.54 | 1863814.64 |
| 2001 | Electronics | 92598527.85 | 92174665.18 | 423862.67 |
| 2002 | Electronics | 94303831.84 | 92598527.85 | 1705303.99 |
+-----+-----+-----+-----+-----+
8 rows in set (0.16 sec)
```

示例 2：使用窗口函数计算 3 天股价均价

假设有如下股票数据，股票代码为 JDR，closing_price 是每天的收盘价：

```
CREATE TABLE stock_ticker (
  stock_symbol STRING,
  closing_price DECIMAL(8, 2),
  closing_date DATETIME
);
```

```
INSERT INTO stock_ticker VALUES
  ("JDR", 12.86, "2014-10-02 00:00:00"),
  ("JDR", 12.89, "2014-10-03 00:00:00"),
  ("JDR", 12.94, "2014-10-04 00:00:00"),
  ("JDR", 12.55, "2014-10-05 00:00:00"),
  ("JDR", 14.03, "2014-10-06 00:00:00"),
  ("JDR", 14.75, "2014-10-07 00:00:00"),
  ("JDR", 13.98, "2014-10-08 00:00:00");
```

```
SELECT * FROM stock_ticker ORDER BY stock_symbol, closing_date;
```

- stock_symbol: JDR; closing_price: 12.86; closing_date: 2014-10-02 00:00:00
- stock_symbol: JDR; closing_price: 12.89; closing_date: 2014-10-03 00:00:00

- stock_symbol: JDR; closing_price: 12.94; closing_date: 2014-10-04 00:00:00
- stock_symbol: JDR; closing_price: 12.55; closing_date: 2014-10-05 00:00:00
- stock_symbol: JDR; closing_price: 14.03; closing_date: 2014-10-06 00:00:00
- stock_symbol: JDR; closing_price: 14.75; closing_date: 2014-10-07 00:00:00
- stock_symbol: JDR; closing_price: 13.98; closing_date: 2014-10-08 00:00:00

下面的查询使用窗口函数生成 moving_average 列，其值为「前一天、当前以及后一天」三天的股价均价。第一天没有前一天的值、最后一天没有后一天的值，因此这两行实际只参与两天的均值计算。这里 PARTITION BY 没有起到实际分组作用（因为所有数据都属于 JDR），但当存在多只股票时，PARTITION BY 可以保证窗口计算只在同一只股票内部进行：

SELECT

```
stock_symbol,  
closing_date,  
closing_price,  
avg(closing_price) OVER (  
    PARTITION BY stock_symbol  
    ORDER BY closing_date  
    ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING  
) AS moving_average  
FROM stock_ticker;
```

- stock_symbol: JDR; closing_date: 2014-10-02 00:00:00; closing_price: 12.86;
moving_average: 12.87
- stock_symbol: JDR; closing_date: 2014-10-03 00:00:00; closing_price: 12.89;
moving_average: 12.89
- stock_symbol: JDR; closing_date: 2014-10-04 00:00:00; closing_price: 12.94;
moving_average: 12.79
- stock_symbol: JDR; closing_date: 2014-10-05 00:00:00; closing_price: 12.55;
moving_average: 13.17
- stock_symbol: JDR; closing_date: 2014-10-06 00:00:00; closing_price: 14.03;
moving_average: 13.77
- stock_symbol: JDR; closing_date: 2014-10-07 00:00:00; closing_price: 14.75;
moving_average: 14.25
- stock_symbol: JDR; closing_date: 2014-10-08 00:00:00; closing_price: 13.98;
moving_average: 14.36

附录：示例数据准备

本文中聚合函数、报告函数、LAG/LEAD 等示例都基于 TPC-DS 风格的表

(item、store_sales、date_dim、customer_address)。如需复现，可按以下步骤准备数据。

1. 创建示例表

```
CREATE DATABASE IF NOT EXISTS doc_tpcds;
```

```
USE doc_tpcds;
```

```
CREATE TABLE IF NOT EXISTS item (  
  i_item_sk bigint not null,  
  i_item_id char(16) not null,  
  i_rec_start_date date,  
  i_rec_end_date date,  
  i_item_desc varchar(200),  
  i_current_price decimal(7,2),  
  i_wholesale_cost decimal(7,2),  
  i_brand_id integer,  
  i_brand char(50),  
  i_class_id integer,  
  i_class char(50),  
  i_category_id integer,  
  i_category char(50),  
  i_manufact_id integer,  
  i_manufact char(50),  
  i_size char(20),  
  i_formulation char(20),  
  i_color char(20),  
  i_units char(10),  
  i_container char(10),  
  i_manager_id integer,  
  i_product_name char(50)  
)  
DUPLICATE KEY(i_item_sk)  
DISTRIBUTED BY HASH(i_item_sk) BUCKETS 12  
PROPERTIES (  
  "replication_num" = "1"  
);
```

```

CREATE TABLE IF NOT EXISTS store_sales (
  ss_item_sk bigint not null,
  ss_ticket_number bigint not null,
  ss_sold_date_sk bigint,
  ss_sold_time_sk bigint,
  ss_customer_sk bigint,
  ss_cdemo_sk bigint,
  ss_hdemo_sk bigint,
  ss_addr_sk bigint,
  ss_store_sk bigint,
  ss_promo_sk bigint,
  ss_quantity integer,
  ss_wholesale_cost decimal(7,2),
  ss_list_price decimal(7,2),
  ss_sales_price decimal(7,2),
  ss_ext_discount_amt decimal(7,2),
  ss_ext_sales_price decimal(7,2),
  ss_ext_wholesale_cost decimal(7,2),
  ss_ext_list_price decimal(7,2),
  ss_ext_tax decimal(7,2),
  ss_coupon_amt decimal(7,2),
  ss_net_paid decimal(7,2),
  ss_net_paid_inc_tax decimal(7,2),
  ss_net_profit decimal(7,2)
)
DUPLICATE KEY(ss_item_sk, ss_ticket_number)
DISTRIBUTED BY HASH(ss_item_sk, ss_ticket_number) BUCKETS 32
PROPERTIES (
  "replication_num" = "1"
);

```

```

CREATE TABLE IF NOT EXISTS date_dim (
  d_date_sk bigint not null,
  d_date_id char(16) not null,
  d_date date,
  d_month_seq integer,
  d_week_seq integer,
  d_quarter_seq integer,
  d_year integer,
  d_dow integer,

```



```

d_moy integer,
d_dom integer,
d_qoy integer,
d_fy_year integer,
d_fy_quarter_seq integer,
d_fy_week_seq integer,
d_day_name char(9),
d_quarter_name char(6),
d_holiday char(1),
d_weekend char(1),
d_following_holiday char(1),
d_first_dom integer,
d_last_dom integer,
d_same_day_ly integer,
d_same_day_lq integer,
d_current_day char(1),
d_current_week char(1),
d_current_month char(1),
d_current_quarter char(1),
d_current_year char(1)
)
DUPLICATE KEY(d_date_sk)
DISTRIBUTED BY HASH(d_date_sk) BUCKETS 12
PROPERTIES (
  "replication_num" = "1"
);

```

```

CREATE TABLE IF NOT EXISTS customer_address (
  ca_address_sk bigint not null,
  ca_address_id char(16) not null,
  ca_street_number char(10),
  ca_street_name varchar(60),
  ca_street_type char(15),
  ca_suite_number char(10),
  ca_city varchar(60),
  ca_county varchar(30),
  ca_state char(2),
  ca_zip char(10),
  ca_country varchar(20),
  ca_gmt_offset decimal(5,2),

```

```

    ca_location_type char(20)
)
DUPLICATE KEY(ca_address_sk)
DISTRIBUTED BY HASH(ca_address_sk) BUCKETS 12
PROPERTIES (
    "replication_num" = "1"
);

```

2. 下载并通过 Stream Load 导入数据

在终端执行如下命令，下载数据到本地，并使用 Stream Load 的方式加载数据：

```
curl -L https://cdn.selectdb.com/static/doc_ddl_dir_d27a752a7b.tar -o - | tar -Jxf -
```

```

curl --location-trusted \
  -u "root:" \
  -H "column_separator:|" \
  -H "columns: i_item_sk, i_item_id, i_rec_start_date, i_rec_end_date, i_item_desc,
i_current_price, i_wholesale_cost, i_brand_id, i_brand, i_class_id, i_class, i_category_id,
i_category, i_manufact_id, i_manufact, i_size, i_formulation, i_color, i_units, i_container,
i_manager_id, i_product_name" \
  -T "doc_ddl_dir/item_1_10.dat" \
  http://127.0.0.1:8030/api/doc_tpcds/item/_stream_load

```

```

curl --location-trusted \
  -u "root:" \
  -H "column_separator:|" \
  -H "columns: d_date_sk, d_date_id, d_date, d_month_seq, d_week_seq, d_quarter_seq,
d_year, d_dow, d_moy, d_dom, d_qoy, d_fy_year, d_fy_quarter_seq, d_fy_week_seq,
d_day_name, d_quarter_name, d_holiday, d_weekend, d_following_holiday, d_first_dom,
d_last_dom, d_same_day_ly, d_same_day_lq, d_current_day, d_current_week,
d_current_month, d_current_quarter, d_current_year" \
  -T "doc_ddl_dir/date_dim_1_10.dat" \
  http://127.0.0.1:8030/api/doc_tpcds/date_dim/_stream_load

```

```

curl --location-trusted \
  -u "root:" \
  -H "column_separator:|" \
  -H "columns: ss_sold_date_sk, ss_sold_time_sk, ss_item_sk, ss_customer_sk,
ss_cdemo_sk, ss_hdemo_sk, ss_addr_sk, ss_store_sk, ss_promo_sk, ss_ticket_number,
ss_quantity, ss_wholesale_cost, ss_list_price, ss_sales_price, ss_ext_discount_amt,

```

```
ss_ext_sales_price, ss_ext_wholesale_cost, ss_ext_list_price, ss_ext_tax, ss_coupon_amt,  
ss_net_paid, ss_net_paid_inc_tax, ss_net_profit" \  
-T "doc_ddl_dir/store_sales.csv" \  
http://127.0.0.1:8030/api/doc_tpcds/store_sales/_stream_load
```

```
curl --location-trusted \  
-u "root:" \  
-H "column_separator:|" \  
-H "columns: ca_address_sk, ca_address_id, ca_street_number, ca_street_name,  
ca_street_type, ca_suite_number, ca_city, ca_county, ca_state, ca_zip, ca_country,  
ca_gmt_offset, ca_location_type" \  
-T "doc_ddl_dir/customer_address_1_10.dat" \  
http://127.0.0.1:8030/api/doc_tpcds/customer_address/_stream_load
```

数据文件

item_1_10.dat、date_dim_1_10.dat、store_sales.csv、customer_address_1_10.dat 也可以从 此压缩包 下载。